Atelier 3 - Agent d'exploration

Vous affichez en temps-réel l'exploration du labyrinthe par un algorithme grâce aux évènements d'avancement de cet algorithme.

Étape 1 - Évènement d'initialisation

Créer une classe `StartEventArgs` dérivé de EventArgs possédant deux propriétés `X` et `Y`.

L'utiliser pour définir un évènement `StartPositionFound` dans `AsciiParser` levé à chaque rencontre d'un 'x' dans le texte. Retirer le paramètre `startPosition` et simplifier le parser.

Utiliser cet évènement depuis le labyrinthe pour initialiser la position de départ.

Vérifier que les tests passent toujours, puis historiser votre version (commit).

Étape 2 - Explorateur

Écrire une classe d'exploration dont le constructeur reçoit un `ICrawler`.

Écrire une méthode `GetOut(int n)` qui effectue, dans un premier temps, des déplacements (appels à `Walk` et `Turn`...) aléatoires et s'arrête dés qu'une tuile `Outside` est atteinte ou après n déplacements.

Si vous écrivez des tests (non obligatoire), il faudra mocker la génération aléatoire pour rendre votre test déterministe.

Historiser cette étape (commit).

Étape 3 - Couche présentation

Pour visualiser les déplacements sans revoir la logique de `GetOut` :

- créer une classe `CrawlingEventArgs` dérivé de EventArgs avec les propriétés `X`, `Y` et `Direction`
- dans votre classe, créer les évènements `PositionChanged` et `DirectionChanged`
- déclencher ces évènements depuis votre classe

Dans le programme principale, afficher le labyrinthe et vous abonner aux évènements pour actualiser la position/orientation de l'explorateur (`^`, `>`, `v` ou `<`) dans le labyrinthe grâce à la fonction `Console.SetCursorPosition`.

Si vous avez fait des tests à l'étape 2, les modifier pour qu'il vérifient le bon déclenchement des évènements avec les bons arguments.

Historiser cette étape (commit).

Remise

Sur moodle le lien vers le github du projet (si modifié).